

A Novel Cooperative Co-evolutionary Framework for Large-scale Overlapping Problems

^{1st} Mingke Chen

*Key Laboratory of Smart Manufacturing in Energy
Chemical Process, Ministry of Education
Engineering Research Center of Process System
Engineering, Ministry of Education
East China University of Science and Technology
Shanghai, China
mk.chen.ecust@gmail.com*

^{2nd} Wei Du*

*Key Laboratory of Smart Manufacturing in Energy
Chemical Process, Ministry of Education
Engineering Research Center of Process System
Engineering, Ministry of Education
East China University of Science and Technology
Shanghai, China
duwei0203@gmail.com*

Abstract—Large-scale overlapping problems are a type of challenging problems because of the special structure of the shared variables. In order to solve the large-scale overlapping problems effectively, the paper proposes a novel cooperative co-evolutionary framework for large-scale overlapping problems called the cooperative co-evolution algorithm with subgroup merging and resource allocation for overlapping problems (CCMRO). In CCMRO, a contribution-based subcomponent merging strategy is proposed to deal with the decomposition of the large-scale overlapping problem and reduce the interaction of subgroups. In addition, a new computational resource allocation strategy (CCRO) is designed to maintain a balance between the full allocation of computational resources and the stability of search speed. Finally, the effectiveness of the proposed CCMRO is verified by the large-scale overlapping benchmark functions.

Index Terms—cooperative co-evolution, large-scale global optimization, overlapping problems, problem decomposition, resource allocation

I. INTRODUCTION

Most real-world optimization problems involving lots of decision variables are called large-scale global optimization (LSGO) problems [1]. Evolutionary algorithms suffer from the curse of dimensionality [2], which means as the dimension of the problem increases, the performance of these algorithms decreases. Cooperative co-evolutionary (CC) algorithms [3] are an effective method for solving LSGO problems. CC algorithms use a divide-and-conquer strategy to reduce optimization complexity by decomposing large-scale problems into smaller subproblems.

The overlapping problem is a special type of LSGO problem. The overlapping problem is represented by a function with overlapping subgroups. The subgroups of the problem overlap with their adjacent counterparts to a certain extent. Large-scale overlapping problems are found in many real-world applications [4]–[7]. For overlapping problems, there are two main challenges that need to be addressed. First, in terms

of decomposition, due to the existence of the shared variables, any decomposition of overlapping problems will result in interaction between subgroups. This puts forward higher requirements for the decomposition of overlapping problems. Second, due to the interaction between subgroups, the computational resource allocation strategy designed for partially separable functions is no longer applicable to overlapping problems. Therefore, the computational resource allocation strategy for overlapping problems needs to be redesigned.

In recent years, many methods have been proposed for decomposing overlapping problems. In overlapping problems, there are direct and indirect interactions between variables due to the presence of shared variables. To decompose overlapping problems, the structure of the overlapping problem should be identified at first. Differential grouping (DG) [8] applies the perturbation to identify the interaction of variables, which can detect the direct interaction relatively accurately. But DG fails to identify the indirect interaction of variables, leading to the failure of identifying the interaction structure of overlapping problems. A large number of DG variants [9]–[12] are proposed to identify indirect interactions. These algorithms can effectively improve the accuracy of interaction recognition and reduce the computational resources. Although they can effectively identify the structure of overlapping problems, all variables of the problems are grouped, which increases the difficulty of solving these overlapping problems.

Many efforts have been made to design more effective decomposition methods for overlapping problems. Recursive differential grouping 3 (RDG3) [13] is one of these algorithms, that breaks the links of multiple subgroups sharing variables by setting thresholds for subgroup sizes. This method shows for the first time that the overlapping problem can benefit from a divide-and-conquer strategy. However, the disconnection of variables mainly depends on the order of variable recognition, which means the method of breaking chains has much room for improvement. Differential grouping with spectral clustering (DGSC) [14] uses spectral clustering to group overlapping problems according to the strength of variable interactions, but the spectral clustering method consumes a huge amount of

*Corresponding author.

computational resources due to the dependence on difference data. Another method called contribution-based cooperative co-evolution for overlapping problems (CBCCO) [15] addresses the overlapping problem by proposing a contribution-based grouping approach that assigns the shared variables to subgroups according to the value of contribution. However, CBCCO still suffers from the interference problem that the optimization of one subgroup can greatly affect other subgroups. Therefore, more attempts should be made to develop more powerful methods for overlapping problems.

After the problem is decomposed, different subgroups may have different importance to the overall problem. Several new CC frameworks have been proposed in order to maximize the use of the computational resources. Most of the existing computational resource allocation strategies are proved to be greedy, which means they continuously optimize the subgroup with the largest contribution until it stagnates or the contribution ranking decreases [16] [17]. In overlapping problems, the contribution of a subgroup may be influenced by other subgroups due to the presence of the shared variables, which means it is not appropriate to adopt a greedy strategy. RDG3 evaluates the efficiency of the contribution-based cooperative co-evolution algorithm for the first time for overlapping problems. In each evolutionary cycle, the component that contributes the most to the overall fitness improvement is selected and evolved. Experiments have shown that the contribution-based cooperative co-evolutionary (CBCC) algorithm is less effective than the traditional CC algorithm for overlapping problems due to the interaction between variables. However, CBCC1 has shown [17] that for non-separable problems (overlapping problems), it is necessary to maintain a high frequency of subgroup collaboration due to the interaction between subgroups, which is beneficial to maintain a stable optimization of the algorithm. CBCCO rewards the important subcomponents based on the round-robin structure. However, without considering the balance between the high collaboration frequency of subgroups and the adequate allocation of computational resources, the algorithm cannot make each subgroup get sufficient computational resources in the cycle.

Based on the above discussions, a novel CC framework called the cooperative co-evolution algorithm with subgroup merging and resource allocation for overlapping problems (CCMRO) is proposed to solve large-scale overlapping problems. The framework proposes a contribution-based subgroup merging strategy to mitigate the impact of interactions among subgroups. In addition, a new computational resource allocation strategy (CCRO) is proposed to compensate for the shortage of subgroup resource allocation. The strategy maintains a balance between the adequate allocation of computational resources and stable optimization speed. The main contributions of this paper are as follows:

- 1) A novel contribution-based subgroup merging strategy is proposed to merge subgroups with similar contributions together, which reduces the influence of interactions between subgroups and improves the optimization efficiency by decreasing the number of interacting subgroups

among overlapping problem subgroups.

- 2) The CCRO is designed to maintain a balance between the full allocation of computational resources and the stability of search speed.
- 3) The proposed CCMRO is compared with the state-of-the-art algorithms on overlapping benchmark functions, and a large number of experiments show that CCMRO can effectively decompose and optimize the overlapping problems.

The rest of the paper is organized as follows. Section II states the background information. Section III introduces the details of the proposed framework. The experimental results are reported in Section IV, together with the analysis of the results. Finally, Section V concludes this paper.

II. BACKGROUND

A. Types of LSGO Problems

There are four kinds of LSGO problems in terms of structure, as shown in Fig. 1. Fig. 1(a) is the fully separable function. Fig. 1(b) is the partially separable function. Both types of the functions can be separated into smaller subgroups, which reduces the difficulty of optimization. Fig. 1(c) shows the overlapping problem studied in this paper, where the two components $\{x_1, x_2, x_3, x_4\}$ and $\{x_3, x_4, x_5, x_6\}$ have shared variables x_3 and x_4 . Fig. 1(d) shows the fully nonseparable problem. Both types cause interactions between subgroups no matter how they are decomposed.

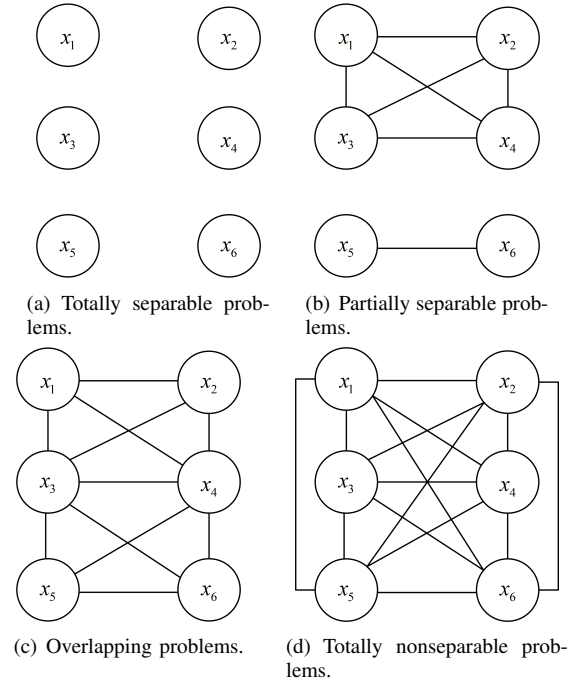


Fig. 1. Illustration of four kinds of LSGO problems.

B. Identification of the Structure of the Overlapping Problem

Before decomposing overlapping problems, the structure of overlapping problems should be identified first. There are di-

rect, indirect, and independent relationships among variables. These relationships are illustrated by the definitions below.

Definition 1 (direct interaction): In an objective function $f(\vec{x})$, decision variables x_i and x_j interact directly with each other if there exists a candidate solution $\vec{x}_*$, denoted by $x_i \leftrightarrow x_j$.

$$\left. \frac{\partial^2 f}{\partial x_i \partial x_j} \right|_{\vec{x}_*} \neq 0 \quad (1)$$

Definition 2 (indirect interaction): Decision variables x_i and x_j interact indirectly with each other if for all candidate solutions, and there exists a set of decision variable $\{x_{k1}, \dots, x_{kt}\} \subset \vec{X}$, denoted by $x_i \leftrightarrow x_{k1} \leftrightarrow \dots \leftrightarrow x_{kt} \leftrightarrow x_j$.

$$\frac{\partial^2 f}{\partial x_i \partial x_j} = 0 \quad (2)$$

Definition 3 (independence): Decision variables x_i and x_j are independent with each other if for all candidate solutions, (3) holds and there no exists a set of decision variable $\{x_{k1}, \dots, x_{kt}\} \subset \vec{X}$, denoted by $x_i \leftrightarrow x_{k1} \leftrightarrow \dots \leftrightarrow x_{kt} \leftrightarrow x_j$.

From Fig. 1(c), there are both direct and indirect interactions in the overlapping problem, e.g. x_1 and x_3 have direct interactions and x_1 and x_5 have indirect interactions.

DG [8] can accurately identify the interaction between variables by means of perturbation with the following equations.

$$|\Delta_{x_i, x_j}[f] - (\Delta_{x_i}[f] + \Delta_{x_j}[f])| > \varepsilon \quad (3)$$

$$\Delta_{x_i, x_j}[f] = f(x_i + \delta_i, x_j + \delta_j) - f(x_i, x_j) \quad (4)$$

$$\Delta_{x_i}[f] = f(x_i + \delta_i, x_j) - f(x_i, x_j) \quad (5)$$

$$\Delta_{x_j}[f] = f(x_i, x_j + \delta_j) - f(x_i, x_j) \quad (6)$$

where $\Delta_{x_j}[f]$ refers to the forward difference of f with respect to variable x_j with interval δ_j . The choice of ε affects the sensitivity of DG in detecting the interactions between the variables.

However, DG cannot identify indirect interaction variables because the interaction variables will be removed from the detection list after each cycle of identification. Recently, a series of DG variants such as global differential grouping (GDG) [19] and differential grouping 2 (DG2) [18] can identify the structure of the overlapping problem by constructing the global matrix of interactions between variables. These algorithms only roughly separate the direct and indirect interaction variables together in a group, resulting in the overlapping problem not being decomposed.

III. THE PROPOSED FRAMEWORK

In order to solve the shortcomings of the existing CC framework for the decomposition and optimization of overlapping problems, we propose a novel CC framework called the cooperative co-evolution algorithm with subgroup merging and resource allocation for overlapping problems (CCMRO) to solve overlapping problems. The pseudocode of the proposed CCMRO algorithm is provided in Algorithm 1. It includes two main phases: 1) the decomposition and merging phase, 2) the computational resource allocation phase. The decomposition

and merging phase can be further divided into two subphases: 1) the shared variable allocation phase, 2) the subgroup merging phase.

Algorithm 1 CCMRO Algorithm

Input: $f(\mathbf{X})$ – the problem $f(\mathbf{X})$
 $\mathbf{X}$ – the variable set $\mathbf{X} = \{x_1, \dots, x_n\}$
 N – the number of test generation N
 K – the *rewardList* optimization K times

Output: $\mathbf{x}^*$ – the context vector $\mathbf{x}^*$

- 1: Interaction matrix $\Lambda = DG2(f(\mathbf{X}))$
- 2: Get the overlapping subcomponent set (**OGS**)
- 3: **OGS** $\rightarrow$ a non-shared variable subgroup set (**GS**) and an overlapping set (**OS**) containing all shared variables by CBCCO [15]
- 4: Set P optimizers $\{O_1, \dots, O_P\}$ corresponding to P sub-components by **GS**
- 5: Initialize the contribution vector $\boldsymbol{\eta} = \{\eta_1, \dots, \eta_P\} = \mathbf{0}$
- 6: Initialize the context vector $\mathbf{x}^* = \mathbf{0}$
- 7: **for** $i=1 \rightarrow P$ **do**
- 8: $f^o = f(\mathbf{x}^*)$
- 9: **for** $j=1 \rightarrow N$ **do**
- 10: Optimize $f(x_i)$ by O_i
- 11: Update $\mathbf{x}^*$
- 12: **end for**
- 13: $\eta_i = f^o - f(\mathbf{x}^*)$
- 14: **end for**
- 15: Update $\boldsymbol{\eta}$
- 16: Get the contribution ranking of subgroup $\mathbf{I} = \{I_1, \dots, I_P\}$ by $\boldsymbol{\eta}$
- 17: **for all** $OGS_{i \cap j}$ in **OS** **do**
- 18: **if** $\eta_i > \eta_j$ **then**
- 19: $NGS_i = GS_i \cup OGS_{i \cap j}$
- 20: **else**
- 21: $NGS_j = GS_j \cup OGS_{i \cap j}$
- 22: **end if**
- 23: **end for**
- 24: /* The shared variable allocation phase */
- 25: [**NGS**] = *Subgroup_Merging* (**NGS**, **OGS**, $\mathbf{I}$, P) /* The subgroup merging phase */
- 26: [$\mathbf{x}^*$] = *Computational_Resource_Allocation* ($f(\mathbf{X})$, $\mathbf{x}^*$, **NGS**) /* The computational resource phase */
- 27: **return** $\mathbf{x}^*$

A. The Decomposition and Merging Phase

1) The shared variable allocation phase: First, we identify the structure of the overlapping problems by the global variable interaction matrix detected by DG2 [18]. We can get the overlapping subcomponent set (**OGS** = $\{OGS_1, \dots, OGS_P\}$) while the shared variables exist in different subgroups. The decomposition of CBCCO [15] is used to divide the set (**OGS**) into a non-shared variable subgroup set (**GS** = $\{GS_1, \dots, GS_P\}$) and an overlapping set (**OS** = $\{OGS_{i \cap j} | 1 \leq i < j \leq P\}$) containing all shared variables. Second, subgroups in the subset of non-shared variables are optimized for 100 generations. The optimized generation is called the test generation and is used to detect the contributions of all subgroups in the **GS**. Since the shared variables are not considered, directly optimizing each subcomponent for 100 generations is identical to using the RR (round-robin) optimization method. The contribution of each subcomponent accumulates in this stage:

$$\eta_i = (f^0 - f^1) + (f^1 - f^2) + \dots + (f^{N-1} - f^N) = f^0 - f^N \quad (7)$$

where N represents the number of the generation.

CBCCO obtains the contribution of each subgroup in the **GS** through optimization, and allocates the shared variables in the **OS** to subgroups in the **GS** with large contributions and interactions, keeping the context vector x^* . Then we get the new subgroup set $\mathbf{NGS} = \{NGS_1, \dots, NGS_P\}$.

2) The subgroup merging phase: In the subgroup merging stage, since the number of the shared variables is small, we regard the contribution of subgroups in the **GS** as the contribution of subgroups in the **NGS**. We rank the contributions of subgroups of **NGS**, and merge the subgroups with similar contributions and the same shared variables. When subgroups are merged, the number of interacting subgroups among overlapping problem subgroups decreases. The pseudo-code is shown in Algorithm 2. In this stage, the subgroup

Algorithm 2 *Subgroup_Merging* ($\mathbf{NGS}$, $\mathbf{OGS}$, I , P)

Input: $\mathbf{NGS}$ – the subgroup set $\mathbf{NGS}$
OGS – the overlapped subgroup set $\mathbf{OGS}$
I – the contribution ranking of subgroup I
P – the number of subgroups of $\mathbf{OGS}$
Output: $\mathbf{NGS}$ – the subgroup set $\mathbf{NGS}$ (subgroup merging)

```

1: Initialize the vector  $M$ 
2: for  $i=2 \rightarrow P$  do
3:   for  $j=1 \rightarrow i-1$  do
4:     while  $OGS_i \cap OGS_j \neq \emptyset$  do
5:       if  $|I_i - I_j| \leq 4$  and  $|NGS_i| \cup |NGS_j| \leq 90$  then
6:         Merge  $i, j$  into  $M$  /*  $M$  is used to store the subgroup
           label that meets the subgroup merging condition */
7:       end if
8:       if  $M$  has the same element then
9:         Found the same element  $M_{a1}$  and  $M_{a2}$ ,  $M_b$  is an element
           that is merged into vector  $M$  simultaneously with  $M_{a1}$ ,  $M_c$ 
           is an element that is merged into vector  $M$  simultaneously
           with  $M_{a2}$  /*  $M_{a1}$ ,  $M_{a2}$ ,  $M_b$ , and  $M_c$  are the labels of
           the subgroups. An example of the order of arrangement in
           vector  $M$ :  $\{M_{a1}, M_b, M_{a2}, M_c\}$  */
10:      end if
11:      if  $I_{M_b} > I_{M_c}$  then
12:        Delete  $M_c$ ,  $M_{a2}$  from  $M$ 
13:      else
14:        Delete  $M_b$ ,  $M_{a1}$  from  $M$ 
15:      end if
16:    end while
17:  end for
18: end for
19: for  $i=1 \rightarrow \frac{|M|}{2}$  do
20:   Merge  $NGS_{M_{2i-1}}$  into  $NGS_{M_{2i}}$ 
21:   Delete  $NGS_{M_{2i-1}}$  from  $\mathbf{NGS}$ 
   /* Subgroup merging */
22: end for
23: return  $\mathbf{NGS}$ 

```

whose contribution rank is smaller than or equal to 4 in the two subgroups with which the shared variable interacts is merged (lines 5-7, 19-22). In addition, the dimensions of subgroups should not be too large. Therefore, the size of the merged subgroup needs to be limited. If the size of the merged subgroup is larger than 90, no merging is performed (lines 5-7). When a subgroup has two subgroups with similar contributions and the same shared variables, there is bound to be a situation where two of the subgroups have a large difference in contributions. It is not possible to merge all the subgroups together, two subgroups with the most similar contributions should be merged at this time, while the other subgroups do not merge (lines 8-15). We get the new subgroup

Overlapping problem:

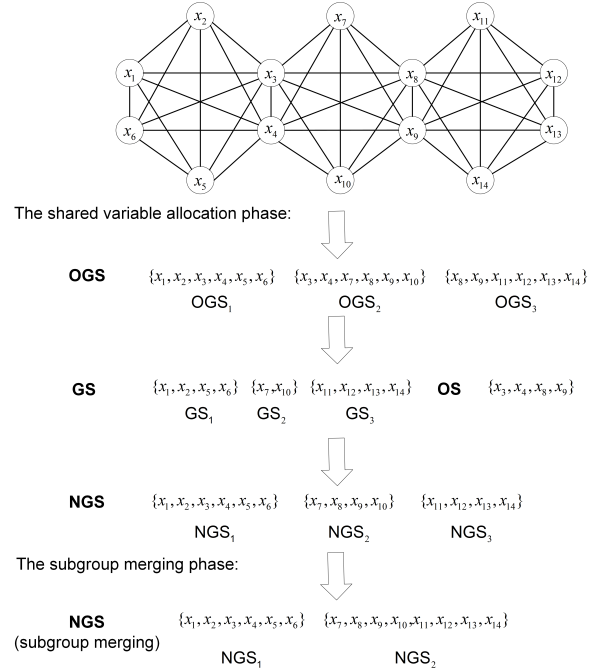


Fig. 2. An example to illustrate the complete process of the decomposition and merging phase.

set (subgroup merging) $\mathbf{NGS} = \{\Theta_1, \dots, \Theta_M\}$, M represents the number of subgroups after the subgroup merging.

We give an example to illustrate the complete process of the decomposition and merging phase, as shown in Fig. 2. In the constructed overlapping problem, $\{x_3, x_4, x_8, x_9\}$ are the shared variables. First, in the shared variable allocation phase, we use the global variable interaction matrix to obtain **OGS** and then divide **OGS** into **GS** and **OS** according to the CBCCO algorithm. We assume that after **GS** is optimized, the contribution of GS_1 is greater than that of GS_2 , and the contribution of GS_2 is greater than that of GS_3 . At this point, we allocate $\{x_3, x_4\}$ to GS_1 and $\{x_8, x_9\}$ to GS_2 (as shown in **NGS**). In the subgroup merging phase, we assume that the contribution rankings of GS_2 and GS_3 are similar. Subgroups with the same shared variable and similar contributions are merged according to the ranking of **GS** subgroup contributions (as shown in **NGS** (subgroup merging)). Considering a special case, if the contribution rankings of GS_1 and GS_2 are similar, and the contribution rankings of GS_2 and GS_3 are also similar, then the contribution rankings of GS_1 and GS_3 may differ significantly. Therefore, only two subgroups with a small contribution ranking gap need to be merged.

B. The Computational Resource Allocation Phase

Experimental studies in the literature [17] have shown that a higher collaboration frequency helps to maintain a steady speed of the algorithm for optimization when the subgroups are not independent of each other. The RR structure has a high frequency of collaboration because each subgroup updates the context vector in each cycle. CBCCO [15] increases the

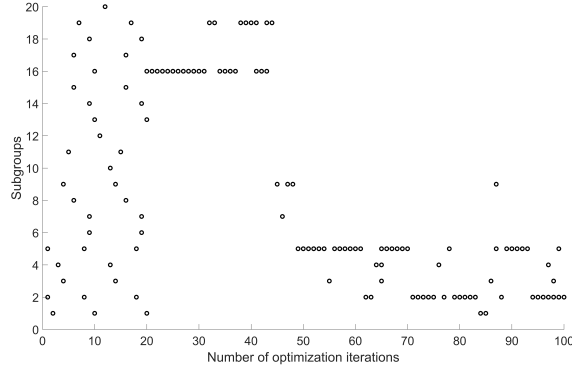


Fig. 3. Reward list subgroup changes in CBCCO

contribution-based reward list based on sequential optimization. CBCCO adds subgroups with contribution value greater than half of the maximum contribution value to the reward list using the decay contribution metric for an additional update generation. However, to pursue high collaboration frequency, the adopted RR structure updates each subgroup for only one generation in one cycle. The subgroup of the reward list cannot get enough computational resources in one cycle.

In addition, the metric of decaying contribution focuses more on the change of subgroup contribution in contemporary times. Fig. 3 shows the distribution of reward list subgroups in the first 100 generations, indicated by circles, and the vertical coordinates are 1-20 subgroups. It can be found that the subgroups of the reward list change very frequently, which is not beneficial for the subgroups with high contributions to get sufficient computational resources in real-time. To maintain the balance between the sufficient allocation of computational resources and the stability of search speed, we propose a new computational resource allocation strategy (CCRO). The pseudo-code is shown in Algorithm 3.

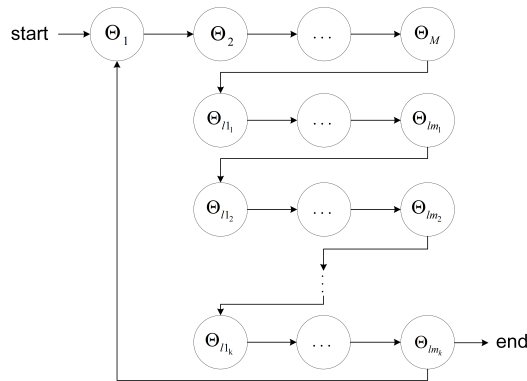


Fig. 4. The structure of CCRO.

First, all subgroups are optimized in RR style (lines 3-8), and the contribution of each subgroup is updated using the

decay contribution metric as in Eq. (8).

$$\eta_i^{g+1} = \frac{\eta_i^g + (f^g - f^{g+1})}{2} \quad (8)$$

where g represents the number of the generation.

Second, according to CBCCO, due to the contribution-calculation method and the awarding method, the historical contribution halves in each generation according to Eq. (8). When the subgroup with the largest contribution does not improve the worst case of the target value at all in the additional generations rewarded, its contribution will be updated directly to the half of its previous value. Therefore, if a subcomponent's contribution value before the reward is less than half of the maximum contribution among subcomponents $\eta_{\max}$, then it definitely cannot be added to the reward list. So, the subgroup with more than half of the maximum contribution is added to the reward list (lines 11-15). The structure of CCRO is shown in Fig. 4. $\{\Theta_1, \dots, \Theta_M\}$ represents the sequential optimization of all subgroups. $\{\Theta_{l1}, \dots, \Theta_{lm1}\}$ is the reward list, and i indicates the number of optimization of the reward list, which is optimized K times in each cycle (lines 16-23). The contribution is also updated.

Algorithm 3 *Computational_Resource_Allocation*
($f(\mathbf{X})$, $\mathbf{x}^*$, NGS)

Input: $f(\mathbf{X})$ – the problem $f(\mathbf{X})$
 $\mathbf{x}^*$ – the context vector $\mathbf{x}^*$
 NGS – the group set NGS (subgroup merging)
 K – the *rewardList* optimization K times

Output: $\mathbf{x}^*$ – the context vector $\mathbf{x}^*$

```

1: Reset optimizers  $\{O_1, \dots, O_M\}$ 
2: Reinitialize the contribution vector  $\boldsymbol{\eta} = \{\eta_1, \dots, \eta_M\} = \mathbf{0}$ 
3: while stop criterion is not met do
4:   for  $i=1 \rightarrow M$  do
5:      $f^o = f(\mathbf{x}^*)$ 
6:     Optimize  $f(x_i)$  by  $O_i$ 
7:     Update  $\mathbf{x}^*$ 
8:      $\eta_i = (\eta_i + (f^o - f(\mathbf{x}^*))) / 2$ 
9:   end for
10:   $\eta_{\max} = \max(\boldsymbol{\eta})$ 
11:  rewardList =  $\emptyset$ 
12:  for  $i=1 \rightarrow M$  do
13:    if  $\eta_{\max} / \eta_i < 2$  then
14:      Add  $i$  into rewardList
15:    end if
16:  end for
17:  for rewardList optimization  $K$  times do
18:    for all  $i$  in rewardList do
19:       $f^o = f(\mathbf{x}^*)$ 
20:      Optimize  $f(x_i)$  by  $O_i$ 
21:      Update  $\mathbf{x}^*$ 
22:       $\eta_i = (\eta_i + (f^o - f(\mathbf{x}^*))) / 2$ 
23:    end for
24:  end for
25: end while
26: return  $\mathbf{x}^*$ 

```

IV. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experimental Settings

First, we introduce the benchmark functions. There are two overlapping functions in the CEC2013 LSGO [22] benchmark functions, f_{13} and f_{14} . f_{13} is a conforming overlapping function, where the optimal values of shared variables are the

TABLE I
LARGE-SCALE OVERLAPPING BENCHMARK CONSISTS OF 8
FUNCTIONS

Func.	Character	Group Size	Elementary Function
f_1	Conforming	50×5+25×10+100×5	$f_{schwefel}(\mathbf{X}) = \sum_{i=1}^D (\sum_{j=1}^i x_j)^2$
f_2	Conflicting	50×5+25×10+100×5	
f_3	Conforming	50×20	
f_4	Conflicting	50×20	
f_5	Conforming	50×5+25×10+100×5	$f_{elliptic}(\mathbf{X}) = \sum_{i=1}^D 10^{6 \frac{i-1}{D-1}} x_i^2$
f_6	Conflicting	50×5+25×10+100×5	
f_7	Conforming	50×20	
f_8	Conflicting	50×20	

same in different subgroups. f_{14} is a conflicting overlapping function, where the optimal values of shared variables differ in different subgroups. To specifically investigate the overlapping problem, the extended benchmark function approach proposed in [23] is used. The benchmark functions consider not only different elementary functions, but also functions with uniform and non-uniform subgroup sizes. From Table I, it can be seen that in addition to the schwefel function, the elliptic elementary function is also extended. Also, uniform and non-uniform subgroup sizes are considered in the benchmark functions. f_{13} and f_{14} in CEC2013 LSGO correspond to f_1 and f_2 in Table I, respectively.

In CCMRO, the covariance matrix adaptive evolution strategy (CMA-ES) is chosen as the optimizer because of its excellent ability [19] to solve low-dimensional continuous optimization problems. The parameters of CMA-ES are set according to the commonly used settings [24]. 30 independent runs were performed for each algorithm to obtain statistical information, including mean and standard deviation. Meanwhile, a two-sided Wilcoxon rank sum test was performed to check if the CCMRO framework significantly outperforms other algorithms in comparison. The significance level was set at 0.05. The maximum number of fitness evaluations (FEs) was used as a stopping criterion for all the algorithms. It was set to 3E+06 according to the recommendations of the benchmark [22]. The number of times to optimize the reward list in each cycle (K) is set to 20.

To examine the performance of CCMRO, it is compared with these algorithms, including the competitive swarm algorithm (CSO) [25], success-history-based adaptive differential evolution with iterative local search (SHADEILS) [26], Dynamic cooperative coevolution (DCC) [27], RDG3 [13], and CBCCO [15]. CSO and SHADEILS are not CC-based algorithms, and their focus is on improving algorithm exploration or development. RDG3, DCC, and CBCCO are proposed for overlapping problems. Among them, SHADEILS and RDG3 are the winners of the CEC 2018 and CEC 2019 LSGO competitions, respectively.

B. Comparisons with State-of-the-Art Algorithms

From the results in Table II, our proposed CCMRO achieves significant advantages in all functions in comparison with CSO, DCC, and SHADEILS. The CC algorithm achieves good results in large-scale overlapping problems. CCMRO has a

better performance in most of the functions compared with RDG3 and CBCCO, and has worse or similar performance in only a small number of conflicting overlapping functions. The results show the effectiveness of our proposed CCMRO in large-scale overlapping problems.

C. Analysis of the Contribution-based Subgroup Merging Strategy

To examine the effectiveness of the contribution-based subgroup merging strategy, contribution-based grouping without subgroup merging and CCMRO are used to examine the effectiveness of the contribution-based subgroup merging strategy. The grouping results of the contribution-based subgroup merging strategy are shown in Table III, and subgroups with similar contributions are merged together. The results of CCRO and CCMRO are shown in Table IV.

For conforming overlapping functions, the contribution-based subgroup merging strategy is significantly better than the contribution-based grouping without subgroup merging. However, for conflicting overlapping functions, the contribution-based subgroup merging strategy only shows similar results compared with the contribution-based grouping strategy without merging subgroups. The results show that merging subgroups with similar contributions effectively reduces the grouping of shared variables with the interacting subgroups. The new strategy facilitates better coordination in the subgroups, but the conflicting nature of the shared variables of the conflicting overlapping functions makes the cooperation of subgroups difficult.

D. Analysis of the New Computational Resource Allocation Strategy

To examine the effectiveness of the proposed new computational resource allocation strategy compared with three other computational resource allocation strategies. The CC-based frameworks: 1) CBCC1 [20]; 2) CCFR [21]; and 3) resource allocation strategy in CBCCO (CBO) [15]. CBCC1 uses the RR structure as the basis. CCFR is a state-of-the-art framework. CBO is a newly proposed computational resource allocation strategy for the overlapping problem. In this experiment, the decomposition method in CBCCO is adopted without merging subgroups, and the results are shown in Table V.

The results show that CCRO is better than CCFR in all functions. This is because CCFR does not adopt the RR structure, which cannot maintain the stable optimization of the algorithm. In addition, the results of CCRO are significantly better than those of CBCC1 and CBO. This shows that the newly proposed resource allocation strategy fully allocates the computational resources of subgroups while maintaining a high cooperation frequency.

E. Parameter Sensitivity Study

In the proposed CCMRO, there is a unique parameter K . K represents the number of times to optimize the reward list in each cycle. The performance of different K was analyzed

TABLE II
THE RESULTS OF THE BENCHMARK FUNCTIONS.

Func.	Stats	CSO	RDG3	SHADEILS	CBCCO	DCC	CCMRO
f_1	Mean	7.50E+08(+)	5.85E+04(+)	9.31E+05(+)	6.10E+01(+)	7.00E+07(+)	9.43E-06
	Std.	3.28E+08	8.85E+04	7.80E+05	6.61E+01	1.27E+08	2.37E-05
f_2	Mean	4.86E+09(+)	4.42E+06(=)	7.98E+06(+)	4.42E+06(=)	7.71E+08(+)	4.41E+06
	Std.	4.96E+09	6.88E+04	9.05E+05	4.33E+04	3.70E+09	5.01E+04
f_3	Mean	6.40E+08(+)	6.26E-02(+)	2.22E+05(+)	4.93E-08(+)	2.24E+07(+)	1.36E-17
	Std.	2.30E+08	1.42E-01	2.03E+05	1.48E-07	3.33E+07	4.28E-17
f_4	Mean	7.58E+09(+)	4.11E+06(=)	5.23E+06(+)	4.08E+06(=)	8.66E+08(+)	4.08E+06
	Std.	3.63E+09	8.30E+04	3.74E+05	8.60E+04	2.54E+09	7.37E+04
f_5	Mean	3.25E+11(+)	2.01E+09(+)	2.82E+10(+)	8.10E+06(+)	8.16E+10(+)	1.15E+04
	Std.	2.61E+10	5.06E+08	6.57E+09	3.36E+06	1.95E+10	7.62E+03
f_6	Mean	3.10E+12(+)	9.36E+08(+)	1.78E+11(+)	8.15E+08(+)	1.03E+12(+)	8.13E+08
	Std.	4.00E+11	1.53E+08	3.62E+10	3.37E+06	3.51E+11	8.45E+06
f_7	Mean	6.91E+11(+)	5.70E+07(+)	5.68E+10(+)	7.22E+04(+)	2.56E+11(+)	4.39E+03
	Std.	6.66E+10	6.14E+07	1.37E+10	3.95E+04	5.66E+10	2.71E+03
f_8	Mean	1.06E+13(+)	2.90E+11(+)	4.45E+11(+)	2.35E+08(-)	3.00E+12(+)	2.64E+08
	Std.	1.17E+12	1.14E+11	1.12E+11	4.63E+06	1.15E+12	6.96E+06
W/T/L		8/0/0	6/2/0	8/0/0	5/2/1	8/0/0	

TABLE III
CONTRIBUTION-BASED GROUP MERGING RESULTS

Func.	Subgroup Size Status	Number of subgroups after subgroup merging
f_1	nonuniform	Sixteen Groups
f_2	nonuniform	Seventeen Groups
f_3	uniform	Fifteen Groups
f_4	uniform	Seventeen Groups
f_5	nonuniform	Sixteen Groups
f_6	nonuniform	Sixteen Groups
f_7	uniform	Fifteen Groups
f_8	uniform	Seventeen Groups

TABLE IV
THE RESULTS OF THE COMPARISON BETWEEN CCMRO AND CCRO

Func	Stats	CCRO	CCMRO
f_1	Mean	6.12E-05(+)	9.43E-06
	Std.	1.27E-04	2.37E-05
f_2	Mean	4.41E+06(=)	4.41E+06
	Std.	4.97E+04	5.01E+04
f_3	Mean	2.00E-14(+)	1.36E-17
	Std.	1.01E-13	4.28E-17
f_4	Mean	4.08E+06(=)	4.08E+06
	Std.	6.48E+04	7.37E+04
f_5	Mean	6.59E+04(+)	1.15E+04
	Std.	2.72E+04	7.62E+03
f_6	Mean	8.13E+08(=)	8.13E+08
	Std.	7.58E+06	8.45E+06
f_7	Mean	1.73E+04(+)	4.39E+03
	Std.	8.25E+03	2.71E+03
f_8	Mean	2.35E+08(=)	2.64E+08
	Std.	3.15E+06	6.96E+06
W/T/L		4/4/0	

by taking different parameters of 1, 20, and 30 for K . In this experiment, the decomposition method in CBCCO is adopted without merging subgroups. The results are shown in Table VI. The best results are obtained when K is equal to 20, and the same structure as CBCCO for $K = 1$. From the results, optimizing fewer generations for the subgroups of the reward

TABLE V
THE RESULTS OF THE COMPARISON AMONG CBCC1, CCFR, CBO, AND CCRO

Func	Stats	CBCC1	CCFR	CBO	CCRO
f_1	Mean	1.18E+03(+)	1.29E+09(+)	6.10E+01(+)	6.12E-05
	Std.	1.24E+03	3.16E+09	6.61E+01	1.27E-04
f_2	Mean	4.40E+06(=)	4.76E+10(+)	4.42E+06(=)	4.41E+06
	Std.	4.11E+04	7.79E+10	4.33E+04	4.97E+04
f_3	Mean	5.00E-06(+)	1.32E+09(+)	4.93E-08(+)	2.00E-14
	Std.	2.68E-05	3.72E+09	1.48E-07	1.01E-13
f_4	Mean	4.09E+06(=)	9.49E+09(+)	4.08E+06(=)	4.08E+06
	Std.	7.30E+04	3.47E+10	8.60E+04	6.48E+04
f_5	Mean	9.79E+08(+)	1.21E+12(+)	8.10E+06(+)	6.59E+04
	Std.	4.86E+08	3.32E+12	3.36E+06	2.72E+04
f_6	Mean	9.70E+08(+)	5.12E+13(+)	8.15E+08(+)	8.13E+08
	Std.	4.08E+08	9.89E+13	3.37E+06	7.58E+06
f_7	Mean	4.04E+07(+)	6.70E+11(+)	7.22E+04(+)	1.73E+04
	Std.	7.88E+07	1.70E+12	3.95E+04	8.25E+03
f_8	Mean	3.17E+08(+)	1.02E+14(+)	2.35E+08(=)	2.35E+08
	Std.	1.59E+08	2.17E+14	4.63E+06	3.15E+06
W/T/L		6/2/0	8/0/0	5/3/0	

list does not sufficiently allocate computational resources for the subgroups. This way is not conducive to the collaborative optimization of the subgroups. The result at $K = 30$ is not as effective as $K = 20$. When the reward list subgroups are optimized too many times in each cycle, which is not conducive to the collaboration of subgroups due to the effect of their optimization on other subgroups. Therefore, we choose $K = 20$ as the number of times to optimize the reward list in each cycle.

V. CONCLUSION AND FUTURE WORK

In this paper, we proposed a novel CC framework called the cooperative co-evolution algorithm with subgroup merging and resource allocation for overlapping problems (CCMRO) for solving large-scale overlapping problems. The two components of CCMRO, the contribution-based subgroup merging strategy and the new computational resource allocation

TABLE VI
THE RESULTS OF CCMRO AT DIFFERENT K VALUES.

Func	$K = 1$	$K = 5$	$K = 20$	$K = 30$
f_1	6.10E+01	1.00E-01	6.12E-05	2.42E-04
f_2	4.42E+06	4.42E+06	4.41E+06	4.44E+06
f_3	4.93E-08	1.32E-11	2.00E-14	9.34E-16
f_4	4.08E+06	4.08E+06	4.08E+06	4.12E+06
f_5	8.10E+06	1.12E+05	6.59E+04	6.74E+04
f_6	8.15E+08	8.15E+08	8.13E+08	8.13E+08
f_7	7.22E+04	1.99E+04	1.73E+04	2.49E+04
f_8	2.35E+08	2.35E+08	2.35E+08	2.35E+08

strategy (CCRO), have shown competitive performance in decomposition and optimization problems, respectively. The contribution-based subgroup merging method can bring significant improvements compared to other decomposition methods. CCRO also offers advantages in optimization speed compared to other CC optimization frameworks. By combining these two components together, the effectiveness and efficiency of CCMRO are further enhanced. The proposed CCMRO has better performance in solving large-scale overlapping problems compared with CSO, SHADEILS, RDG3, CBCCO, DCC, and other state-of-the-art algorithms.

In the future, we will investigate the decomposition and optimization of conflicting overlapping subgroups.

ACKNOWLEDGMENTS

This work was supported by National Natural Science Foundation of China (Key Program: 62136003), National Natural Science Foundation of China (62173144), National Natural Science Foundation of Shanghai (21ZR1416100), Shanghai Rising-Star Program (22QA1402400), Fundamental Research Funds for the Central Universities and the Shanghai AI Lab.

REFERENCES

- [1] Mei Y, Li X, Yao X. Cooperative coevolution with route distance grouping for large-scale capacitated arc routing problems[J]. IEEE Transactions on Evolutionary Computation, 2013, 18(3): 435-449. Yang Z, Tang K, Yao X. Large scale evolutionary optimization using cooperative coevolution[J]. Information sciences, 2008, 178(15): 2985-2999.
- [2] Omidvar M, Li X, Yao X. A review of population-based metaheuristics for large-scale black-box global optimization: Part A[J]. IEEE Transactions on Evolutionary Computation, 2021.
- [3] Potter M A, Jong K A D. A cooperative coevolutionary approach to function optimization[C]//International conference on parallel problem solving from nature. Springer, Berlin, Heidelberg, 1994: 249-257.
- [4] Martins J, Lambe A B. Multidisciplinary Design Optimization: A Survey of Architectures[J]. AIAA Journal, 2013, 51(9):2049-2075.
- [5] Yassine A, Braha D. Complex Concurrent Engineering and the Design Structure Matrix Method[J]. Concurrent Engineering, 2003, 11(3):165-176.
- [6] raun R D, Kroo I M. Development and Application of the Collaborative Optimization Architecture in a Multidisciplinary Design Environment[M]. NASA Langley Technical Report Server, 1995.
- [7] illagra A, Pandolfi D, Leguizamón G, et al. Optimization of potable water networks with hybrid metaheuristics[C]// 2017 XVII Workshop on Information Processing and Control (RPIC). IEEE, 2017.
- [8] Omidvar M N, Li X, Mei Y, et al. Cooperative Co-Evolution With Differential Grouping for Large Scale Optimization[J]. IEEE Transactions on Evolutionary Computation, 2014, 18(3):378-393.
- [9] Mei Y, Omidvar M N, Li X, et al. A competitive divide-and-conquer algorithm for unconstrained large-scale black-box optimization[J]. ACM Transactions on Mathematical Software (TOMS), 2016, 42(2): 1-24.
- [10] Sun Y, Kirley M, Halgamuge S K. Extended Differential Grouping for Large Scale Global Optimization with Direct and Indirect Variable Interactions[J]. ACM, 2015:313-320.
- [11] Sun Y, Kirley M, Halgamuge S K. A Recursive Decomposition Method for Large Scale Continuous Optimization[J]. IEEE Transactions on Evolutionary Computation, 2017:1-1.
- [12] Yang M, Zhou A, Li C, et al. An Efficient Recursive Differential Grouping for Large-Scale Continuous Problems[J]. IEEE Transactions on Evolutionary Computation, 2020, PP(99):1-1.
- [13] Y.Sun, X.Li, A. Ernst, and M. N. Omidvar, "Decomposition for largescale optimization problems with overlapping components," in Proc.IEEE Congr. Evol. Comput. (CEC), Wellington, New Zealand, 2019, pp. 326–333.
- [14] Li L, Fang W, Wang Q, et al. Differential Grouping with Spectral Clustering for Large Scale Global Optimization[C]// 2019 IEEE Congress on Evolutionary Computation (CEC). IEEE, 2019.
- [15] Jia Y H, Mei Y, Zhang M. Contribution-Based Cooperative Co-Evolution for Nonseparable Large-Scale Problems With Overlapping Subcomponents[J]. IEEE Transactions on Cybernetics, 2020, PP(99).
- [16] M.N.Omidvar, B. Kazimipour, X. Li, and X. Yao, "Cbcc3—A contribution-based cooperative co-evolutionary algorithm with improved exploration/exploitation balance," in Proc. IEEE Congr. Evol. Comput.(CEC), Vancouver, BC, Canada, 2016, pp. 3541–3548.
- [17] B.Kazimipour, M. N. Omidvar, X. Li, and A. K. Qin, "A sensitivity analysis of contribution-based cooperative co-evolutionary algorithms," in Proc. IEEE Congr. Evol. Comput. (CEC), Sendai, Japan, 2015, pp. 417–424.
- [18] Omidvar M N, Ming Y, Yi M, et al. DG2: a faster and more accurate differential grouping for large-scale black-box optimization[J]. IEEE Transactions on Evolutionary Computation, 2019, 21(6):929-942.
- [19] Yi, Mei, Mohammad, et al. A Competitive Divide-and-Conquer Algorithm for Unconstrained Large-Scale Black-Box Optimization[J]. ACM Transactions on Mathematical Software (TOMS), 2016, 42(2).
- [20] Omidvar M N, Li X, Xin Y. Smart Use of Computational Resources Based on Contribution for Cooperative Co-evolutionary Algorithms[C]// 13th Annual Genetic and Evolutionary Computation Conference, GECCO 2011, Proceedings, Dublin, Ireland, July 12-16, 2011. 2011.
- [21] M.Yang et al., "Efficient resource allocation in cooperative co-evolution for large-scale global optimization," IEEE Trans. Evol. Comput., vol. 21, no. 4, pp. 493–505, Aug. 2017.
- [22] X.Li, K.Tang, M.N.Omidvar, Z.Yang, K.Qin, and H.China, "Benchmark functions for the CEC'2013 special session and competition on large-scale global optimization," Gene, vol. 7, no. 33, p. 8, 2013.
- [23] Y.-H. Jia, Y.-R. Zhou, Y. Lin, W.-J. Yu, Y. Gao, and L. Lu, "A distributed cooperative co-evolutionary CMA evolution strategy for global optimization of large-scale overlapping problems," IEEE Access, vol. 7, pp. 19821–19834, 2019.
- [24] N.Hansen, "The CMA evolution strategy: A comparing review," in Towards a New Evolutionary Computation. Heidelberg, Germany: Springer, 2006, pp. 75–102.
- [25] R.Cheng and Y.Jin, "A competitive swarm optimizer for large scale optimization," IEEE Trans. Cybern., vol. 45, no. 2, pp. 191–204, Feb. 2015.
- [26] D.Molina, A.LaTorre, and F. Herrera, "SHADE with iterative local search for large-scale global optimization," in Proc. Congr. Evol.Comput. (CEC), Rio de Janeiro, Brazil, 2018, pp. 1–8.
- [27] Zhang X Y, Gong Y J, Ying L, et al. Dynamic Cooperative Coevolution for Large Scale Optimization[J]. IEEE Transactions on Evolutionary Computation, 2019, 23(6):935-948.